The above configuration is also directing the container to create a database named *glarimy* since the *AddService* is configured to use that database.

However, this is still not sufficient. The MySQL container writes the data on to the file system that is mapped to the container. Once the container is restarted, the files are gone! That is not good for us. We want the data to be written on to the disk in such a way that it outlives the containers. In other words, we want to mount a volume so that the container uses only that mount point. The last line in the above configuration is meant for that.

The following is the resulting full manifest in *docker-compose.yml:*

```
version: "2"
networks:
  glarimy:
    driver: bridge
services:
  zookeeper:
    image: docker.io/bitnami/zookeeper:3.8
    ports:
      - "2181:2181"
    volumes:
      - "zookeeper_data:/glarimy"
    environment:
      - ALLOW_ANONYMOUS_LOGIN=yes
    networks:
      - glarimy
  kafka:
    image: docker.io/bitnami/kafka:3.1
    ports:
      - "9092:9092"
    volumes:
      - "kafka_data:/glarimy"
    environment:
      - KAFKA_CFG_ZOOKEEPER_CONNECT=zookeeper:2181
      - ALLOW_PLAINTEXT_LISTENER=yes
    networks:
      - glarimy
    depends_on:
      - zookeeper
  mysqldb:
    image: mysql:latest
    networks:
      - glarimy
    environment:
      - MYSQL_ROOT_PASSWORD=admin
      - MYSQL_DATABASE=glarimy
    volumes:
      - "mysql_data:/glarimy"
  ums:
    image: glarimy/ums-add-service
    networks:
      - glarimy
    depends_on:
      - zookeeper
      - mysqldb
volumes:
  zookeeper_data:
    driver: local
  kafka_data:
    driver: local
  mysql_data:
    driver: local
```

Run the following command to deploy the containers, like we did in the previous part:

```
$ docker-compose up
```

However, this is not our actual goal. The docker-compose can deploy multiple containers in one go, only on a single machine. It does not offer resilience and does not offer cluster deployment.

Kubernetes and Minikube

The production infrastructure of microservices consists of several machines forming a cluster. The containers are expected to be distributed fairly across the machines (aka nodes) in the cluster. New nodes may be added to the cluster and existing nodes may be removed from it at any time. Yet, the containers are expected to be rescheduled on the current set of nodes.

Kubernetes does take care of such an orchestration. It deploys the containers across the cluster and redistributes them whenever needed — all without any manual intervention.

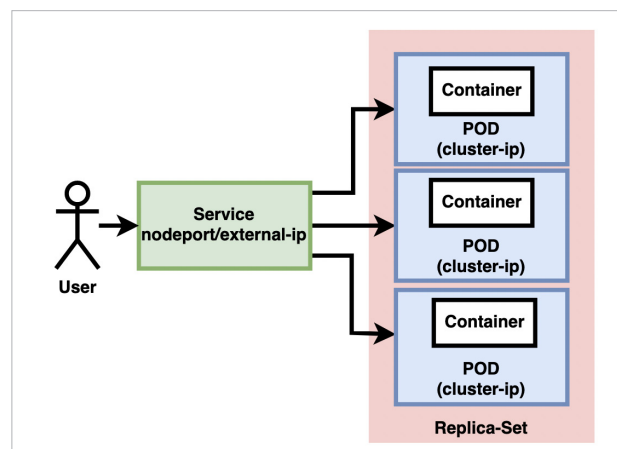

Figure 2: Kubernetes architecture